



兰州工业学院

实 验 报 告

(2023 —2024 学年第 二 学期)

课程名称 单片机原理与接口技术

专业班级 专升本电信23

学 号 ZB2305010116

姓 名 姜 勇

实验项目	实验八 综合实验（考核）		
实验时间	2024年6月12日	实验地点	信息与通信工程基础实验室 (微机系统实验室)

一、实验目的

- 1. 了解液晶显示模块 LCD1602 的工作原理及特点；
- 2. 熟悉 LCD 的使用和编程方法；
- 3. 掌握数字钟的实现方法；
- 4. 综合应用定时器、按键、LCD 等资源的应用。

二、实验环境

硬件：单片机开发板，计算机一台；

软件：Keil uVSION4, Proteus 软件。

二、实验内容

使用定时器实现一个基于 LCD 的简易数字钟，要求具有显示时、分、秒及校时功能。

要求：（1）画出电路图和程序流程图；（2）写出源程序代码。

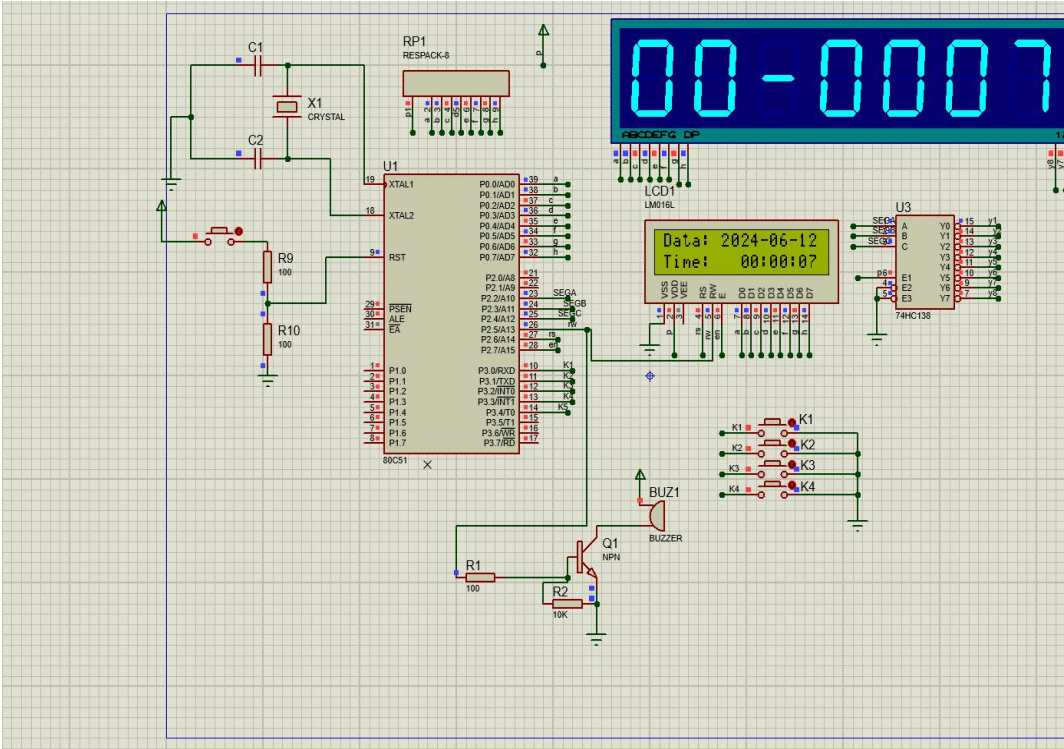


图 1 电路图

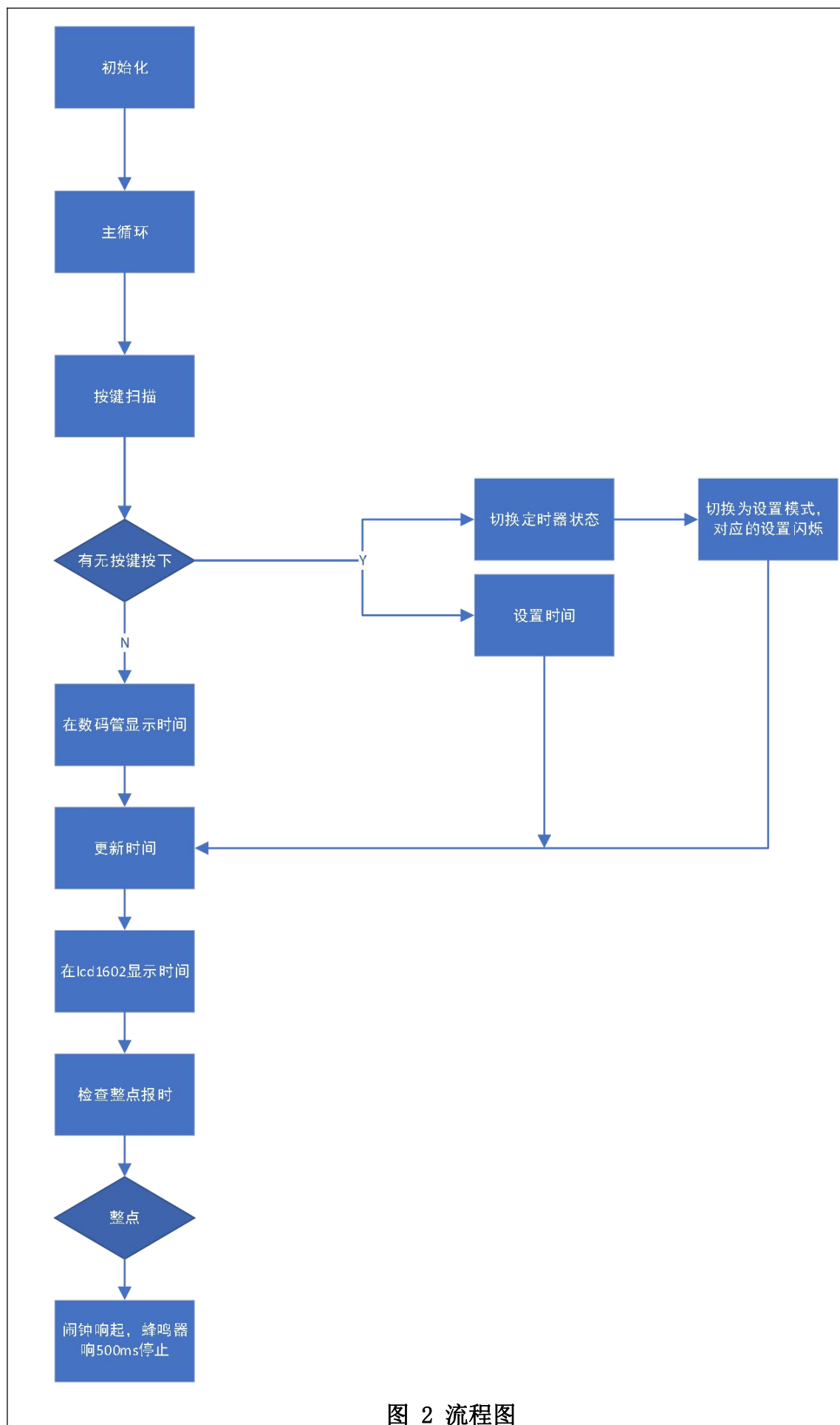


图 2 流程图

//数码管模块

```
#include "seg.h"
```

```
// 初始化数码管显示数据数组
```

```
u8 smg[17] =
```

```
{
```

```
    0x3f, 0x06, 0x5b, 0x4f, 0x66, 0x6d, 0x7d, 0x07,
```

```
    0x7f, 0x6f, 0x77, 0x7c, 0x39, 0x5e, 0x79, 0x71
```

```
};
```

```
// 显示函数，dat 是要显示的数据数组，pos 是起始显示位置
```

```
void display(u8 dat[], u8 pos)
```

```
{
```

```
    u8 i = 0;
```

```
    // 位选的临时变量
```

```
    u8 pos_temp = pos - 1;
```

```
    // 循环遍历要显示的数据
```

```
    for (i = pos_temp; i < 8; i++)
```

```
    {
```

```
        // 根据位选控制数码管的位选引脚
```

```
        switch (i)
```

```
        {
```

```
        case 0:
```

```
            LSC = 1;
```

```
            LSB = 1;
```

```
            LSA = 1;
```

```
            break;
```

```
        case 1:
```

```
            LSC = 1;
```

```
            LSB = 1;
```

```
            LSA = 0;
```

```
            break;
```

```
case 2:
    LSC = 1;
    LSB = 0;
    LSA = 1;
    break;
case 3:
    LSC = 1;
    LSB = 0;
    LSA = 0;
    break;
case 4:
    LSC = 0;
    LSB = 1;
    LSA = 1;
    break;
case 5:
    LSC = 0;
    LSB = 1;
    LSA = 0;
    break;
case 6:
    LSC = 0;
    LSB = 0;
    LSA = 1;
    break;
case 7:
    LSC = 0;
    LSB = 0;
    LSA = 0;
    break;
```

```

    }

    // 在当前位选下，显示对应的数码管段选数据
    seg = dat[i - pos_temp];

    // 延时函数，假设 delay(1) 函数已经定义，提供必要的显示时间
    delay(1);

    // 关闭数码管显示，防止重影
    seg = 0x00;

}

}

//定时器模块
#include "timer.h"

// 外部变量的定义，表示时间和日期
u8 second = 0;
u8 minute = 0;
u8 hour = 0;
u16 year = 2024;
u8 month = 6;
u8 day = 12;
u8 digits[8] = {0}; // 用于显示的数码管数组
bit timer_running = 1; // 定时器运行状态，默认为运行状态
// 切换计时器状态的函数实现
void toggle_timer()
{
    timer_running = !timer_running; // 切换计时器状态
    TR0 = timer_running;           // 启动/停止计时器
}

// 定时器0初始化函数实现
void timer0_init()
{
    TMOD = 0x01; // 设置定时器0为模式1

```

```

    TH0 = (65536 - 50000) / 256;    // 设置初值，实现定时50ms
    TL0 = (65536 - 50000) % 256;

    ET0 = 1;                        // 允许定时器0中断
    TR0 = 1;                        // 启动定时器0
    EA = 1;                         // 开总中断
}

// 定时器0中断服务程序实现
void timer0_ISR() interrupt 1
{
    if (timer_running) // 如果定时器在运行
    {
        static unsigned char count = 0; // 静态计数器，用于控制时间更新频率

        TH0 = (65536 - 50000) / 256;    // 重置定时器初值，实现50ms定时

        TL0 = (65536 - 50000) % 256;

        count++; // 计数器加一

        if (count >= 20) // 每计数到20，即每1秒更新一次时间
        {
            count = 0;

            second++;

            if (second >= 60)
            {
                second = 0;

                minute++;

                if (minute >= 60)
                {
                    minute = 0;

                    hour++;

                    if (hour >= 24)
                    {

```

```

        hour = 0;

        day++;

        if (day > 31)
        {
            day = 1;

            month++;

            if (month > 12)
            {
                month = 1;

                year++;
            }
        }
    }
}

// 更新数码管显示数据
dights[0] = smg[hour / 10];
dights[1] = smg[hour % 10];
dights[2] = 0x40;
dights[3] = smg[minute / 10];
dights[4] = smg[minute % 10];
//    dights[5] = 0x40;
dights[5] = smg[second / 10];
dights[6] = smg[second % 10];
}

//按键模块
#include "key.h"
u8 alarm_setting_active = 0;

```

```

u8 setting_active = 0;      // 设置模式状态，默认为非设置模式
u8 current_setting = 0;     // 当前设置模式，默认为设置年份
u8 saved_alarm_hour = 0;
u8 saved_alarm_minute = 0;
u8 key;
// 月份的天数，非闰年
u8 days_in_month[12] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30,
31};
char key_buf[] = {0x77, 0x7b, 0x7d, 0x7e,
                  0xb7, 0xbb, 0xbd, 0xbe,
                  0xd7, 0xdb, 0xdd, 0xde,
                  0xe7, 0xeb, 0xed, 0xee
                  };// 定义key_buf数组，存储按键扫描的值
char getKey()// 获取按键值的函数
{
    char key_scan[] = {0xbf, 0x7f, 0xef, 0xdf};
    char i = 0, j = 0;
    for (i = 0; i < 4 ; i++)
    {
        P1 = key_scan[i];
        if ((P1 & 0x0f) != 0x0f)
            delay(10);
        for(j=0; j<16; j++)
            if(key_buf[j]==P1)
                return j;
    }
    return -1;
}
// 切换设置模式的函数
void toggle_setting()

```

```

{
    setting_active = !setting_active; // 切换设置模式状态
    if (setting_active)
    {
        current_setting = 0; // 开始设置年
    }
}
// 切换闹钟设置模式
void toggle_alarm_setting() {
    alarm_setting_active = !alarm_setting_active;
    if (alarm_setting_active) {
        // 设置闹钟时，默认从当前时间开始设置
        alarm_hour = saved_alarm_hour;
        alarm_minute = saved_alarm_minute;
    } else {
        saved_alarm_hour = alarm_hour;
        saved_alarm_minute = alarm_minute;
    }
}
// 按键扫描函数实现
void key_scan()
{
    // 如果按键K1被按下
    if (K1 == 0)
    {
        while (!K1)
        {
            delay(10); // 防抖延时
        }

        if (setting_active)

```

```

{
    switch (current_setting)
    {
    case 0: // 设置年
        year++;
        if (year > 9999) year = 0;
        break;
    case 1: // 设置月
        month++;
        if (month > 12) month = 1;
        break;
    case 2: // 设置日
        day++;
        if (day > 31) day = 1;
        break;
    case 3: // 设置小时
        hour++;
        if (hour >= 24) hour = 0;
        break;
    case 4: // 设置分钟
        minute++;
        if (minute >= 60)
        {
            minute = 0;
            hour++;
            if (hour >= 24)
            {
                hour = 0;
                day++;
                if (day > days_in_month[month - 1])

```

```

        {
            day = 1;
            month++;
            if (month > 12)
            {
                month = 1;
                year++;
                if (year > 9999) year = 0;
            }
        }
    }
    break;
case 5: // 设置秒
    second++;
    if (second >= 60)
    {
        second = 0;
        minute++;
        if (minute >= 60)
        {
            minute = 0;
            hour++;
            if (hour >= 24)
            {
                hour = 0;
                day++;
                if (day > days_in_month[month - 1])
                {
                    day = 1;

```

```

        month++;
        if (month > 12)
        {
            month = 1;
            year++;
            if (year > 9999) year = 0;
        }
    }
}

break;
}

} else if (alarm_setting_active) {
    // 设置闹钟小时加1
    alarm_hour++;
    if (alarm_hour > 23) alarm_hour = 0;
}

// 如果按键K2被按下
if (K2 == 0)
{
    while (!K2)
    {
        delay(10); // 防抖延时
    }

    if (setting_active)
    {
        switch (current_setting)

```

```

{
case 0: // 设置年
    year--;
    if (year < 0) year = 9999;
    break;
case 1: // 设置月
    month--;
    if (month < 1) month = 12;
    break;
case 2: // 设置日
    day--;
    if (day < 1) day = 31;
    break;
case 3: // 设置小时
    hour--;
    if (hour < 0) hour = 23;
    break;
case 4: // 设置分钟
    minute--;
    if (minute < 0)
    {
        minute = 59;
        hour--;
        if (hour < 0) hour = 23;
    }
    break;
case 5: // 设置秒
    second--;
    if (second < 0)
    {

```

```

        second = 59;
        minute--;
        if (minute < 0)
        {
            minute = 59;
            hour--;
            if (hour < 0) hour = 23;
        }
    }
    break;
}
} else if (alarm_setting_active) {
    // 设置闹钟小时减1
    alarm_hour--;
    if (alarm_hour < 0) alarm_hour = 23;
}
}
// 如果按键K3被按下
if (K3 == 0)
{
    while (!K3)
    {
        delay(10); // 防抖延时
    }
    if (setting_active)
    {
        // 切换到下一个设置模式
        current_setting++;
        if (current_setting > 5) current_setting = 0; // 循环切换
至设置年

```

```

    } else if (alarm_setting_active) {
        // 设置闹钟分钟加1
        alarm_minute++;
        if (alarm_minute > 59) alarm_minute = 0;
    }
}
// 如果按键K4被按下
if (K4 == 0) {
    while (!K4) {
        delay(10); // 防抖延时
    }
    if (alarm_setting_active) {
        // 如果在闹钟设置模式下，减少闹钟分钟
        alarm_minute--;
        if (alarm_minute < 0) alarm_minute = 59;
    } else {
        // 如果不在闹钟设置模式，切换定时器状态和设置模式
        toggle_timer();
        toggle_setting();
    }
}

key = getKey();
if (key != -1) { // 确保有按键被按下
    if (key == 0) {
        delay(10); // 防抖延时
        if (getKey() == 0) { // 确认按键仍然被按下
            toggle_alarm_setting(); // 切换闹钟设置模式
        }
    }
}
}

```

```

}

//蜂鸣器模块
#include "beep.h"

void beep_init() {
    // 设置蜂鸣器控制引脚为输出模式
    BEEP_PIN = 0; // 默认关闭蜂鸣器
}

void beep_on() {
    BEEP_PIN = 1; // 设置引脚为高电平，触发蜂鸣器
}

void beep_off() {
    BEEP_PIN = 0; // 设置引脚为低电平，关闭蜂鸣器
}

//lcd1602模块
#include "../lcd1602/lcd1602.h"

u8 show_date[16] = {"Date: "}; // 日期显示字符串
u8 show_time[16] = {"Time: "}; // 时间显示字符串
u8 blink_counter = 0; // 用于控制闪烁的计数器

void lcd1602_write_cmd(u8 cmd)
{
    RS = 0; // 命令模式
    RW = 0; // 写模式
    EN = 1; // 使能
    P0 = cmd; // 写入命令
    delay(1); // 等待一段时间
    EN = 0; // 禁止
}

void lcd1602_write_dat(u8 dat)
{

```

```

    RS = 1; // 数据模式
    RW = 0; // 写模式
    EN = 1; // 使能
    P0 = dat; // 写入数据
    delay(1); // 等待一段时间
    EN = 0; // 禁止
}

void lcd1602_init()
{
    lcd1602_write_cmd(0x38); // 8位数据接口, 2行显示, 5*7点阵
    lcd1602_write_cmd(0x0c); // 显示开, 不显示光标, 光标不闪烁
    lcd1602_write_cmd(0x06); // 光标右移, 字符不移动
    lcd1602_write_cmd(0x01); // 清屏
}

void lcd1602_display_time()
{
    int i;
    if (alarm_setting_active) {
        // 显示设置闹钟的状态
        lcd1602_write_cmd(0x80); // 设置显示位置为第一行第一列
        show_time[0] = 'T';
        show_time[1] = 'i';
        show_time[2] = 'm';
        show_time[3] = 'e';
        show_time[4] = ' ';
        show_time[5] = ':';
        show_time[6] = ' ';
        show_time[7] = ' ';
        show_time[8] = hour / 10 + '0';
        show_time[9] = hour % 10 + '0';
    }
}

```

```

show_time[10] = ':';
show_time[11] = minute / 10 + '0';
show_time[12] = minute % 10 + '0';
show_time[13] = ':';
show_time[14] = second / 10 + '0';
show_time[15] = second % 10 + '0';
for (i = 0; i < 16; i++)
{
    lcd1602_write_dat(show_time[i]);
}

lcd1602_write_cmd(0xc0); // 设置显示位置为第二行第一列
show_time[0] = 'a';
show_time[1] = 'l';
show_time[2] = 'a';
show_time[3] = 'r';
show_time[4] = 'm';
show_time[5] = ':';
// 显示设置的闹钟时间
show_time[6] = alarm_hour / 10 + '0';
show_time[7] = alarm_hour % 10 + '0';
show_time[8] = ':';
show_time[9] = alarm_minute / 10 + '0';
show_time[10] = alarm_minute % 10 + '0';
show_time[11] = ' ';
show_time[12] = ' ';
show_time[13] = ' ';
show_time[14] = ' ';
show_time[15] = ' ';
show_time[16] = ' ';
for (i = 0; i < 16; i++) {

```

```

        lcd1602_write_dat(show_time[i]);
    }
} else {
    // 更新时间字符串
    show_time[0] = 'T';
    show_time[1] = 'i';
    show_time[2] = 'm';
    show_time[3] = 'e';
    show_time[4] = ' ';
    show_time[5] = ':';
    show_time[6] = ' ';
    show_time[7] = ' ';
    show_time[8] = hour / 10 + '0';
    show_time[9] = hour % 10 + '0';
    show_time[10] = ':';
    show_time[11] = minute / 10 + '0';
    show_time[12] = minute % 10 + '0';
    show_time[13] = ':';
    show_time[14] = second / 10 + '0';
    show_time[15] = second % 10 + '0';
    // 更新日期字符串
    show_date[6] = year / 1000 + '0';
    show_date[7] = year % 1000 / 100 + '0';
    show_date[8] = year % 100 / 10 + '0';
    show_date[9] = year % 10 + '0';
    show_date[10] = '-';
    show_date[11] = month / 10 + '0';
    show_date[12] = month % 10 + '0';
    show_date[13] = '-';
    show_date[14] = day / 10 + '0';

```

```

show_date[15] = day % 10 + '0';

// 显示当前时间和日期
lcd1602_write_cmd(0xc0); // 设置显示位置为第二行第一列
for (i = 0; i < 16; i++)
{
    lcd1602_write_dat(show_time[i]);
}
lcd1602_write_cmd(0x80); // 设置显示位置为第一行第一列
for (i = 0; i < 16; i++)
{
    lcd1602_write_dat(show_date[i]);
}
// 检查是否需要闪烁
if (setting_active)
{
    switch (current_setting)
    {
        case 0:
            // 年闪烁
            if (blink_counter < 50)
            {
                lcd1602_write_cmd(0x86); // 设置显示位置为年
                lcd1602_write_dat(' '); // 清除第一个数字位
                lcd1602_write_dat(' '); // 清除第二个数字位
                lcd1602_write_dat(' '); // 清除第三个数字位
                lcd1602_write_dat(' '); // 清除第四个数字位
            }
        }
    }
}

```

```

    }
    else
    {
        lcd1602_write_cmd(0x86); // 设置显示位置为年
        for (i = 6; i < 10; i++)
        {
            lcd1602_write_dat(show_date[i]); // 显示
        }
    }
    break;
case 1:
    // 月闪烁
    if (blink_counter < 50)
    {
        lcd1602_write_cmd(0x8B); // 设置显示位置为月
        lcd1602_write_dat(' '); // 清除第一个数字位
        lcd1602_write_dat(' '); // 清除第二个数字位
    }
    else
    {
        lcd1602_write_cmd(0x8B); // 设置显示位置为月
        for (i = 11; i < 13; i++)
        {
            lcd1602_write_dat(show_date[i]); // 显示
        }
    }
}

```

的第一个数字
置
置

的第一个数字
日的两个数字

的第一个数字
置

```
    }  
    break;  
case 2:  
    // 日闪烁  
    if (blink_counter < 50)  
    {  
        lcd1602_write_cmd(0x8E); // 设置显示位置为日  
        lcd1602_write_dat(' '); // 清除第一个数字位  
        lcd1602_write_dat(' '); // 清除第二个数字位  
    }  
    else  
    {  
        lcd1602_write_cmd(0x8E); // 设置显示位置为日  
        for (i = 14; i < 16; i++)  
        {  
            lcd1602_write_dat(show_date[i]); // 显示  
        }  
    }  
    break;  
case 3:  
    // 时闪烁  
    if (blink_counter < 50)  
    {  
        lcd1602_write_cmd(0xc8); // 设置显示位置为时  
        lcd1602_write_dat(' '); // 清除第一个数字位
```

置

的第一个数字

时的两个数字

的第一个数字

置

置

的第一个数字

分的两个数字

```
        lcd1602_write_dat(' '); // 清除第二个数字位  
    }  
    else  
    {  
        lcd1602_write_cmd(0xc8); // 设置显示位置为时  
        for (i = 8; i < 10; i++)  
        {  
            lcd1602_write_dat(show_time[i]); // 显示  
        }  
    }  
    break;  
case 4:  
    // 分闪烁  
    if (blink_counter < 50)  
    {  
        lcd1602_write_cmd(0xcB); // 设置显示位置为分  
        lcd1602_write_dat(' '); // 清除第一个数字位  
        lcd1602_write_dat(' '); // 清除第二个数字位  
    }  
    else  
    {  
        lcd1602_write_cmd(0xcB); // 设置显示位置为分  
        for (i = 11; i < 13; i++)  
        {  
            lcd1602_write_dat(show_time[i]); // 显示
```

```

        }
    }
    break;
case 5:
    // 秒闪烁
    if (blink_counter < 50)
    {
        lcd1602_write_cmd(0xcE); // 设置显示位置为秒
        lcd1602_write_dat(' '); // 清除第一个数字位
        lcd1602_write_dat(' '); // 清除第二个数字位
    }
    else
    {
        lcd1602_write_cmd(0xcE); // 设置显示位置为秒
        for (i = 14; i < 16; i++)
        {
            lcd1602_write_dat(show_time[i]); // 显示
        }
    }
    break;
}

// 增加闪烁计数器，确保计数器在0到99之间循环
blink_counter++;
if (blink_counter >= 100)
{
    blink_counter = 0;
}

```

```

    }

}

//main函数
#include <reg51.h>
#include "../timer/timer.h"
#include "../seg/seg.h"
#include "../key/key.h"
#include "../beep/beep.h"
#include "../lcd1602/lcd1602.h"

void uptime()
{
    // 更新数码管显示数据
    dights[0] = smg[hour / 10];
    dights[1] = smg[hour % 10];
    dights[2] = 0x40;
    dights[3] = smg[minute / 10];
    dights[4] = smg[minute % 10];
    // dights[5] = 0x40;
    dights[5] = smg[second / 10];
    dights[6] = smg[second % 10];
}

void check_hourly_alarm()
{
    // 检查是否整点，触发蜂鸣器
    if (second == 0 && minute == 0)
    {
        // 循环 500 次，每次产生 1 毫秒的延时
        ul6 i;
    }
}

```

```

        for (i = 0; i < 500; i++)
        {
            beep_on(); // 触发蜂鸣器
            delay(1);  // 延时1毫秒
            beep_off(); // 关闭蜂鸣器
        }
    }
}

void check_alarm() {
if (second == 0&& minute==saved_alarm_minute&&hour==saved_alarm_hour )
{
    // 触发闹钟蜂鸣
    ul6 i;
    for (i = 0; i < 500; i++)
    {
        beep_on(); // 触发蜂鸣器
        delay(1);  // 延时1毫秒
        beep_off(); // 关闭蜂鸣器
    }
}
}

void main()
{
    timer0_init();    // 初始化定时器0
    beep_init();      // 初始化蜂鸣器
    lcd1602_init();   // 初始化 LCD1602
    while (1)
    {
        key_scan();           // 扫描键盘
        display(dights, 1);   // 在数码管上显示时间
    }
}

```

```
    uptime();                // 更新数码管显示数据
    lcd1602_display_time();  // 在 LCD1602 上显示时间
    check_alarm();
    check_hourly_alarm();    // 检查是否需要触发整点报警
}
}
```

四、实验总结

实验八 成绩评定表

序号	考核项目	分值分布	成绩
1	出勤与纪律	10	
2	实验任务完成情况	50	
3	实验报告质量	40	
总分			

备注：考核项目及分值分布应依据教学大纲确定。